

Project Part B Report

COMP30024 Artificial Intelligence | Kavish(1616728) & Tavish(1615873)

1. Introduction

This report describes the design and implementation of an AI agent for Cascade based on iterative deepening negamax search with alpha-beta pruning. The agent uses a compact integer array to represent the board, Zobrist hashing to store previously seen positions, and a multi-feature evaluation function.

It also uses move-ordering techniques such as the killer heuristic, history heuristic, and prioritising moves found in the hash table, to search as deeply as possible within a 180-second CPU time limit. A quiescence search is used at the leaf nodes to help the agent look a bit further in situations where important captures are possible.

2. Game Representation and Architecture

2.1 State Representation

The board is stored as a Python list of 64 integers (8x8 grid). Each square encodes both owner and stack height: positive values for Red, negative for Blue, and 0 for empty squares. The absolute value gives stack height, and the sign (+/-) gives ownership.

Board states are copied using `list.copy()` during search. Each state also stores a Zobrist hash (updated via XOR), turn counters, an undo stack for fast backtracking, and a dictionary for repetition detection.

2.2 Move Generation

Moves are generated in `moves.py`. During placement, all empty squares are valid except those restricted near opponent stacks after the first move. During the main phase, moves are generated from all owned stacks.

Each stack can move to adjacent squares, capture weaker enemy stacks, or perform cascade moves if height ≥ 2 . Neighbouring squares are precomputed for efficiency.

The number of legal moves decreases over time, typically starting around 30–50 in the midgame and reducing as stacks are removed, allowing deeper search later in the game..

2.3 Program Architecture

The agent is divided into six Python modules:

- **program.py** (the referee-facing Agent class)
- **state.py** (board representation, apply/undo, hashing, end conditions)
- **moves.py** (legal move generation)
- **search.py** (iterative deepening negamax, transposition table, move ordering)
- **evaluation.py** (static position evaluation)
- **placement.py** (placement phase heuristic).

3. Search Algorithm

3.1 Base Algorithm: Negamax

The agent uses negamax during play, a simplified version of minimax for zero-sum games where one player's gain is the other's loss. This allows the same logic to be used for both players by treating a position's value as the negative of the opponent's value, making the search simpler.

At each step, the agent evaluates all legal moves, applies them, recursively searches the resulting positions, and undoes the move using an undo system. It selects the move with the best score. Terminal states return fixed values: draws return 0, while wins and losses return large positive or negative values, adjusted so faster wins and slower losses are preferred.

3.2 Iterative Deepening

Instead of searching to a fixed depth, the agent uses iterative deepening. This means it searches to depth 1, then depth 2, then depth 3, and so on, until it runs out of time. The best move found at each depth is saved, so if the search is stopped early, the agent still has a valid move to play.

This approach has two main benefits. First, it allows the agent to always return a move even under strict time limits. Second, earlier shallow searches help improve later searches by filling the transposition table and improving move ordering, which means deeper searches become more efficient.

3.3 Alpha-Beta Pruning/PVS

The agent uses alpha-beta pruning in its search to skip branches that cannot improve the final result. This reduces the number of positions that need to be checked and allows the search to go deeper within the same time limit. This works best when good move ordering is used, because strong moves are checked first and weaker ones are cut off earlier.

On top of this, the agent uses Principal Variation Search (PVS). The first move at each position is searched normally using a full window. The remaining moves are first checked using a smaller "quick test" window. If a move looks good enough, it is re-searched with the full window to confirm its value. This reduces the number of full searches and makes the search faster, especially when move ordering is strong.

3.4 Move Ordering

Alpha-beta pruning is very sensitive to the order in which moves are searched: examining strong moves first leads to more cutoffs and reduces the number of positions that need to be evaluated. To take advantage of this, moves are scored and sorted in descending order before each node is expanded using the following priority scheme:

Priority	Move Type	Score Formula	Rationale
1st	Hash move (TT)	10,000,000	Best move from a previous TT lookup
2nd	EAT (capture)	$1,000,000 + V \times 100$	V = victim height; captures win tokens
3rd	CASCADE with threat	$500,000 + T \times 50$	T = sum of enemy heights in cascade path

4th	MOVE (merge)	200,000 + killer bonus + history bonus	Merging builds tall stacks
5th	MOVE (relocate)	0 + killer bonus + history bonus	Non-capturing repositioning

Table 1: Move ordering priority scheme.

3.5 Killer and History Heuristics

On top of the main move ordering system, the agent uses two additional heuristics to improve search efficiency. The killer heuristic keeps track, for each search depth, of up to two MOVE actions that previously caused a beta cutoff. These moves are given a large bonus (90,000 and 80,000) when they appear again at the same depth, because a move that was strong enough to cut off one branch is often strong in similar positions.

The history heuristic tracks how often each MOVE (based on its starting position and direction) has caused a beta cutoff across the whole search. Moves that frequently lead to cutoffs are given higher scores, with stronger rewards for deeper parts of the search. This helps the agent prefer moves that have been consistently useful in many different positions. Both heuristics are stored using simple dictionaries and add very little overhead.

3.6 Transposition Table

The transposition table (TT) is a Python dictionary that stores previously seen positions using a 64-bit Zobrist hash as the key. Each entry stores the search depth, score, type of score (exact value or upper/lower bound), and the best move found from that position. When the agent finds a position in the table, it only reuses the stored score if it was calculated at the same or deeper search depth and is still valid for the current bounds. Otherwise, it still uses the stored best move to help improve move ordering.

3.7 Quiescence Search

At depth 0, the agent continues searching capture moves instead of stopping immediately. It first evaluates the current position (“stand-pat”). If no better capture exists, it returns the score; otherwise, it searches capture moves ordered by capture value. This avoids missing any sharp tactical changes.

3.8 Time Management

The agent manages its 180-second total time budget by adjusting how much time it uses each turn. At the start of each move, it checks the remaining time from the referee, subtracts a safety buffer of 5 seconds, and divides the remaining time by an estimate of how many moves are left (based on game progress).

The time per move is also adjusted depending on the stage of the game. After turn 30, the agent allows slightly more time per move, and after turn 80 it allows even more, since fewer pieces on the board usually means faster search. The final time limit for any move is capped between 50 milliseconds and 8 seconds to prevent both overly short and overly long searches.

Game Phase	Turn Range	Time Multiplier	Rationale
Early	1-30	1.0x	High branching factor; conserve budget
Mid	31-80	1.3x	Fewer stacks; can afford deeper search
Late	81-300	$1.3 \times 1.2 = 1.56x$	Very few stacks; deep search cheaper

Table 2: Adaptive time multipliers by game phase.

4. Evaluation Function and Strategy

4.1 Overview

Because the search cannot reach the true end of the game in all lines, a static evaluation function is used to estimate how good non-final positions are. The function returns a score from Red's point of view (positive values are good for Red, negative values are good for Blue). Inside the negamax framework, this is adjusted so the score is always relative to the player whose turn it is. The final evaluation is a weighted combination of six features added together.

4.2 Feature Descriptions

Material (weight 50): The total height of all tokens for each player. This is the most important feature because winning is based on eliminating opponent tokens. A difference of one token has a large impact on the score, making material the main factor in decision-making.

Stack count (weight 5): The number of separate stacks each player has. Having more stacks usually means more options and threats, while combining everything into one large stack can be powerful but risky because it removes flexibility.

Centre control (weight 1): Tokens placed in the central 4x4 area (rows and columns 2-5) receive a small bonus based on their height. Central positions are stronger because they can reach more neighbouring squares compared to edge positions.

Capture threat (weight 25): Rewards situations where the agent can capture enemy stacks. This encourages immediate attacking opportunities.

Capture risk (weight 35): Measures how exposed the agent's own stacks are to being captured. This is weighted higher than capture threat because losing tokens is more damaging than gaining them is beneficial, since the game is decided by elimination.

Edge vulnerability (weight 6): Penalises stacks that are close to the edge of the board. Edge stacks are more likely to be pushed off the board by cascade moves, which results in losing all tokens in that stack. The penalty is stronger for stacks directly on the edge than for those slightly further in.

Feature	Weight	Formula	Strategic Purpose
Material	100	sum(own heights) - sum(opp heights)	Directly tracks win condition
Stack count	5	own stacks - opp stacks	Measures positional flexibility
Centre control	1	own central heights - opp central	Connectivity and mobility
Capture threat	25	sum(capturable enemy heights)	Offensive pressure
Capture risk	35	sum(capturable own heights)	Defensive urgency
Edge vuln.	6	sum(edge-penalised own heights)	Cascade push vulnerability

Table 3: Evaluation function features and weights.

4.3 Weight Justification

Rather than testing a single fixed set of parameters, we gradually adjusted the weights and evaluated performance through repeated self-play between different versions of the agent.

Since the space of possible weight combinations is continuous and very large, it was not practical to test every option. Instead, we started with a simple baseline based on intuition and then refined the weights step by step based on observed match results, keeping changes that led to more consistent improvements.

This process was repeated until further adjustments no longer produced noticeable gains in performance. At that point, the final set of weights was chosen for evaluation and reporting.

5. Performance Evaluation

5.1 Experimental Setup

Performance was measured by running the agent against several opponents using the provided referee system on a local machine. Each game followed the standard constraints: a 180-second CPU time limit per player and a 250 MB memory limit. To avoid bias from turn order, games were played with the agent acting as both Red and Blue in equal numbers.

5.2 Opponents

Three types of opponents were used for evaluation. The **random agent** chooses a legal move at random each turn and represents a baseline level of performance. The **simple greedy agent** provided in the template selects moves based only on immediate benefit, without looking ahead. The third opponent was an **earlier version** of our own agent, which used the same search method but different evaluation function weights.

5.3 Win Rate Results

Opponent	Games	Wins	Losses	Draws	Win Rate
Random agent	10	10	0	0	100%
Simple / Greedy	10	10	0	0	100%

Table 4: Win/loss/draw results against both opponent types.

Earlier agent configurations are not reported in the results table because each evaluation was performed using different evaluation function weightings. As the baseline agent changed across experiments during parameter tuning, the resulting outcomes are not directly comparable and are therefore excluded from the table.

5.4 Runtime Analysis

Table 5 shows how the search performs in different stages of the game. The branching factor is estimated using $\text{nodes}^{(1/\text{depth})}$. While the estimated branching factor stays fairly similar across phases, the number of nodes and move time decrease in the late game, suggesting the search becomes more efficient as the position simplifies when stacks are removed.

Phase	Turn Range	Avg Depth	Avg Nodes	Est. Branching	Avg Move Time
Early	1–30	4.8 plies	110k	11.8	2.1s
Mid	31–80	4.9 plies	107k	13	2s
Late	81–300	4.7 plies	94k	13.1	1.8s

Table 5: Search depth and node count by game phase.

6. Additional Technical Features

6.1 Zobrist Hashing

The Zobrist hash is built by assigning a random number to every combination of piece type, height, and board position. The full position hash is then created by XOR-ing the values for all occupied squares. When a move is made or undone, the hash is updated in constant time by XOR-ing only the changed squares.

6.2 Placement Phase Heuristic

During the 8-turn placement phase the agent uses a greedy heuristic rather than tree search. Each candidate placement cell is scored on: distance to the board centre (prefer central cells), distance to edge (penalise edge and near-edge cells), distance to the nearest friendly stack, and local enemy pressure (penalise cells adjacent to tall opponent stacks). The highest-scoring legal cell is selected.

6.3 Mate Distance Encoding

End positions are given special scores. A winning position is scored as a very large value minus how deep in the search it was found, and a losing position is scored as a very large negative value plus the depth. This means the agent prefers faster wins over slower wins, and tries to delay losses for as long as possible. Without this adjustment, the agent might not choose the quickest win or could behave poorly in already-winning positions.

6.4 Threefold Repetition Detection in Search

The agent keeps track of how many times each position has been seen during the search. If the same position appears three times, it is treated as a draw and given a score of 0 immediately. This prevents the agent from repeating moves in a loop and risking a draw when a better outcome is possible.

7. Conclusion

The agent for Cascade is built using iterative deepening negamax search with alpha-beta pruning. The agent also uses transposition tables, principal variation search, killer and history heuristics, and quiescence search to improve performance. It represents the board using a compact integer list and uses Zobrist hashing to efficiently store and retrieve previously seen positions. The search is also managed within a strict time limit that adapts depending on the stage of the game.

The evaluation function uses six weighted features that reflect key parts of the game. Material is the most important factor, while capture opportunities, capture risk, and edge weakness also influence decisions. Overall, the design focuses on correctness and reliability, ensuring the agent always returns a legal move even under time pressure, while also applying the standard search optimisations covered in the course.

References:

ChatGPT:

- For finding optimisation techniques not taught in class and how to implement them using code.
- For giving a template for the report.
- For coding help (syntax and debugging)
- For rewording/clarity purposes (in the report).